

May 29, 2026

1 Opgave 1

1.1 Spm 1)

The branch and bound method is used to solve integer linear programming problems (ILP) by iteratively restricting the feasible region of a linear programming relaxation of the problem. The method involves branching on decision variables, creating subproblems, and using bounds to eliminate subproblems that cannot yield better solutions than the best known solution. The process continues until all subproblems have been explored or eliminated, resulting in the optimal integer solution.

```
[7]: import pulp as PLP
from math import ceil, floor
import copy

def branch_and_bound(LPmodel, sense, best_so_far = [None], objectives = [None],
    problem = [0]):

    """
    :param LPmodel:
        input a Linear Programming relaxation of the ILP problem.
        Note it is import to keep track of if it is a minimization or maximization_
    ↪problem, as this will determine the
        branching strategy and pruning strategy.
    :return:
        Iterative Branch and Bound solution to the ILP problem.
    """
    if best_so_far[0] is None:
        if sense == "maximize":
            best_so_far[0] = -10 ** 6
        else:
            best_so_far[0] = 10 ** 6

    if sense != "maximize" and sense != "minimize":
        raise ValueError("sense must be either 'maximize' or 'minimize'")
    if sense == "maximize":
        def objective_is_not_better(obj):
```

```

        return obj < best_so_far[0]
if sense == "minimize":
    def objective_is_not_better(obj):
        return obj > best_so_far[0]

def is_integer_value():
    """
    :return:
        dict where keys are variable names and values are boolean values
    ↪ indicating whether variable is integer.
    """
    eps = 10**-4
    vars = LPmodel.variables()

    d = dict()
    # Assign boolean values to the decision variables based on whether they
    ↪ are integer or not
    for var in vars:
        if abs(var.varValue - ceil(var.varValue)) > eps and abs(var.
    ↪ varValue - floor(var.varValue)) > eps:
            d[var.name] = False
        else:
            d[var.name] = True
    return d
print(20*"#")
print("Problem : ", problem[0])
print(20 * "#")
print()
### Step 1 - Solve problem ###
LPmodel.solve(PLP.PULP_CBC_CMD(msg = 0))
obj = PLP.value(LPmodel.objective)
### Step 2 - Branch on non-integer decision variable
vars = is_integer_value()
if LPmodel.status == PLP.LpStatusInfeasible:
    print("Pruning branch, infeasible\n")
    return objectives
if objective_is_not_better(obj):
    print("Pruning branch with objective value ", obj, " which is worse
    ↪ than best so far ", best_so_far[0])
    print("Decision variables: ")
    for v in LPmodel.variables():
        print(v.name, "=", v.varValue)
    print("Objective value: ", obj)
    print()
    return objectives
if all([v for v in vars.values()]):

```

```

print("Found feasible branch, backtracking")
print("Decision variables: ")
for v in LPmodel.variables():
    print(v.name, "=", v.varValue)
print("Objective value: ", obj)
print()
if obj > best_so_far[0] and sense == "maximize":
    best_so_far[0] = obj
    in_problem = problem[0]
if obj < best_so_far[0] and sense == "minimize":
    best_so_far[0] = obj

objectives[0] = obj
return objectives

for name, value in vars.items():
    if value:
        continue
    else:
        branch_name = name
        branch_value = LPmodel.variablesDict()[branch_name].varValue
        break

### Step 3 - Create two branches and solve recursively ###
### Base case - All decision variables or the problem is not feasible or
↪ objective does not become better ###

# Left branch
left_model = copy.deepcopy(LPmodel)
left_model += left_model.variablesDict()[branch_name] <= floor(branch_value)

for v in LPmodel.variables():
    print(v.name, "=", v.varValue)
print("Objective: ", obj)
print("Adding constraint ", branch_name, " <= ", floor(branch_value), " to ↪
↪ left branch")
print()
problem[0] = problem[0] + 1
branch_and_bound(left_model, sense, best_so_far, objectives, problem)

# Right branch
right_model = copy.deepcopy(LPmodel)
right_model += right_model.variablesDict()[branch_name] >= ↪
↪ ceil(branch_value)
print("Adding constraint ", branch_name, " >= ", ceil(branch_value), " to ↪
↪ left branch")

```

```

print()
problem[0] = problem[0] + 1
branch_and_bound(right_model, sense, best_so_far, objectives, problem)

return best_so_far[0]

model = PLP.LpProblem("Opg1", sense=PLP.LpMaximize)
x1 = PLP.LpVariable("x1", cat=PLP.LpContinuous, lowBound=0)
x2 = PLP.LpVariable("x2", cat=PLP.LpContinuous, lowBound=0)
model += 3*x1 + 2*x2, "Objective"
model += 4*x1 + 2*x2 <= 15
model += 2*x1 + 3*x2 <= 12

branch_and_bound(model, sense="maximize")

```

```

#####
Problem : 0
#####

x1 = 2.625
x2 = 2.25
Objective: 12.375
Adding constraint x1 <= 2 to left branch

```

```

#####
Problem : 1
#####

x1 = 2.0
x2 = 2.6666667
Objective: 11.3333334
Adding constraint x2 <= 2 to left branch

```

```

#####
Problem : 2
#####

Found feasible branch, backtracking
Decision variables:
x1 = 2.0
x2 = 2.0
Objective value: 10.0

Adding constraint x2 >= 3 to left branch

```

```

#####
Problem : 3
#####

```

```

x1 = 1.5
x2 = 3.0
Objective: 10.5
Adding constraint x1 <= 1 to left branch

#####
Problem : 4
#####

Pruning branch with objective value 9.6666666 which is worse than best so far
10.0
Decision variables:
x1 = 1.0
x2 = 3.3333333
Objective value: 9.6666666

Adding constraint x1 >= 2 to left branch

#####
Problem : 5
#####

Pruning branch, infeasible

Adding constraint x1 >= 3 to left branch

#####
Problem : 6
#####

x1 = 3.0
x2 = 1.5
Objective: 12.0
Adding constraint x2 <= 1 to left branch

#####
Problem : 7
#####

x1 = 3.25
x2 = 1.0
Objective: 11.75
Adding constraint x1 <= 3 to left branch

#####
Problem : 8
#####

```

Found feasible branch, backtracking

Decision variables:

x1 = 3.0

x2 = 1.0

Objective value: 11.0

Adding constraint x1 >= 4 to left branch

#####

Problem : 9

#####

Pruning branch, infeasible

Adding constraint x2 >= 2 to left branch

#####

Problem : 10

#####

Pruning branch, infeasible

[7]: (11.0, 10)

1.2 Spm 2.)

We now apply an integer constraint and solve the ILP

```
[2]: modelILP = PLP.LpProblem("Opg1", sense=PLP.LpMaximize)
x1 = PLP.LpVariable("x1", cat=PLP.LpInteger, lowBound=0)
x2 = PLP.LpVariable("x2", cat=PLP.LpInteger, lowBound=0)
modelILP += 3*x1 + 2*x2, "Objective"
modelILP += 4*x1 + 2*x2 <= 15
modelILP += 2*x1 + 3*x2 <= 12
modelILP.solve()
print("Status:", PLP.LpStatus[modelILP.status])
for var in modelILP.variables():
    print(var.name, ":", var.varValue)
print("Objective value:", PLP.value(modelILP.objective))
```

Status: Optimal

x1 : 3.0

x2 : 1.0

Objective value: 11.0

2 Opgave 2)

2.1 Spm. 1)

The critical path is to be understood as the path that we must take, one, such that the path does not break the constraint the all previous tasks/nodes that enter the node/task are completed and that the path is as short as possible.

We start at node 0, then when task A and C is done, we move to node 2 and 3, where B is been completed in the mean time. Node 3 can then be started when E is done, as D is already finished. The final task 4 is then limited by G as F is completed in the time it takes E and G to be finished.

```
[3]: import pulp as PLP

class critical_path_problem:
    """
    This class defines the critical path problem. The implementation follows
    ↪from uge 7 Afsnit 5.3 Netværksmodeller
    slide 42.

    Input:
    Nodes - list of integers
    Edges - list of integers
    times - dict of time associated with each edge

    """

    def __init__(self, nodes, edges, times):
        self.nodes = nodes
        self.edges = edges
        self.times = times

        #Model, decision variable and objective
        self.model = PLP.LpProblem("critical_path_problem", sense=PLP.
        ↪LpMinimize)

        self.z = PLP.LpVariable("z", lowBound=0, cat=PLP.LpContinuous)
        self.t = PLP.LpVariable.dicts("t", range(len(self.nodes)), lowBound=0,
        ↪cat=PLP.LpContinuous)

        self.model += self.z, "Objective"

    def construct_constraints(self):
        for i, j in self.edges:
            # Ending j must be greater than start i + time for activity i to j
            self.model += self.t[j] >= self.t[i] + self.times[(i, j)]
        for j in self.nodes:
```

```

        # decision variable t must be less than or equal to z for all nodes
        self.model += self.t[j] <= self.z
    self.constraints = "ADDED"

def solve_and_print(self):
    if self.constraints != "ADDED":
        raise Exception("You must add constraints before solving")
    self.model.solve()

    print("Status:", PLP.LpStatus[self.model.status])
    print("Objective value:", PLP.value(self.model.objective))

    for j in self.nodes:
        print(f"Node {j} has time {round(self.t[j].varValue, 2)}")
    for var in self.model.variables():
        if "t" in var.name:
            print(var.name, ":", var.value())

# Example usage:
nodes = list(range(5))

# (i,j,cost)
edges_and_cost = [(0,1,8), (0,2,14), (1,2,8), (2,3,8), (1,3,7), (2,4,14),
    ↪ (3,4,7)]
edges = []
cost = {}
for i, j, c in edges_and_cost:
    e = (i,j)
    edges.append(e)
    cost[e] = c
CPP = critical_path_problem(nodes,edges,cost)
CPP.construct_constraints()
CPP.solve_and_print()

```

```

Status: Optimal
Objective value: 31.0
Node 0 has time 0.0
Node 1 has time 8.0
Node 2 has time 16.0
Node 3 has time 24.0
Node 4 has time 31.0
t_0 : 0.0
t_1 : 8.0
t_2 : 16.0
t_3 : 24.0
t_4 : 31.0

```


2.2 Spm 2.)

We can add binary variable indicating whether a time saving has been used for the given prices. We add a constraint such that we don't surpass the budget, and that we can at most reduce the speed up by one of the methods.

We can then incorporate the saving by either 3 or 5 time units in the constraint of the time to begin a new project

```
[14]: nodes = list(range(5))

# (i,j,cost)
edges_and_cost = [(0,1,8), (0,2,14), (1,2,8), (2,3,8), (1,3,7), (2,4,14),
    ↪(3,4,7)]

edges = []
cost = {}
for i, j, c in edges_and_cost:
    e = (i,j)
    edges.append(e)
    cost[e] = c

# Start with a clean CPP model
CPP2 = critical_path_problem(nodes,edges,cost)

# Fix variable names: "x" for x, "y" for y
CPP2.x = PLP.LpVariable.dicts("x", CPP2.edges, cat=PLP.LpBinary)
CPP2.y = PLP.LpVariable.dicts("y", CPP2.edges, cat=PLP.LpBinary)

for i,j in CPP2.edges:
    # Reduce time by 3 if x is chosen, 5 if y is chosen
    if True:
        CPP2.model += CPP2.t[j] >= CPP2.t[i] + CPP2.times[(i,j)] - 3 * CPP2.
    ↪x[(i,j)] - 5 * CPP2.y[(i,j)]
    else:
        CPP2.model += CPP2.t[j] >= CPP2.t[i] + CPP2.times[(i,j)]

for i,j in CPP2.edges:
    # A task can be reduced by at most one of the methods
    CPP2.model += CPP2.x[(i,j)] + CPP2.y[(i,j)] <= 1

CPP2.model += PLP.lpSum(CPP2.x[(i,j)]*2000 + CPP2.y[(i,j)]*4000 for i,j in CPP2.
    ↪edges) <= 10000

for j in CPP2.nodes:
    # decision variable t must be less than or equal to z for all nodes
    CPP2.model += CPP2.t[j] <= CPP2.z
```

```

CPP2.constraints = "ADDED"

# To see CBC output explicitly:
CPP2.model.solve(PLP.PULP_CBC_CMD(msg=1))

print("Status:", PLP.LpStatus[CPP2.model.status])
print("Objective value:", PLP.value(CPP2.model.objective))

for j in CPP2.nodes:
    print(f"Node {j} has time {round(CPP2.t[j].varValue, 2)}")
for i,j in CPP2.edges:
    if CPP2.x[(i,j)].varValue > 0.5:
        print(f"Edge {(i,j)} reduced by x (3 days)")
    if CPP2.y[(i,j)].varValue > 0.5:
        print(f"Edge {(i,j)} reduced by y (5 days)")

```

```

Status: Optimal
Objective value: 23.0
Node 0 has time 0.0
Node 1 has time 5.0
Node 2 has time 11.0
Node 3 has time 16.0
Node 4 has time 23.0
Edge (0, 1) reduced by x (3 days)
Edge (0, 2) reduced by x (3 days)
Edge (1, 2) reduced by x (3 days)
Edge (2, 3) reduced by x (3 days)
Edge (2, 4) reduced by x (3 days)

```

[]:

3 Opgave 3)

3.1 Spm.1)

Since this problem does not consider the passengers that migrate between the planes, it is an assignment problem that minimizes the product of distance and flow. We can form a cost matrix by multiplying the distances and flows from the planes and gates. I use the class below:

```

[24]: import pulp as PLP
import numpy as np
class AssignmentProblem:

    """This implementation follows "Afsnit 5.3 Netværksmodeller" from week 7 """
    def __init__(self, n,
                  cost_matrix = None,
                  name = "AssignmentProblem",
                  ):

```

```

# Number of jobs/assignments
self.n = n
self.name = name
self.cost_matrix = cost_matrix
# Range of decision variable x
self.variable_range = range(self.n)

# decision variables
self.x = PLP.LpVariable.dicts("x",
                              (self.variable_range, self.variable_range),
                              lowBound = 0 )

if self.cost_matrix is None:
    raise ValueError("Cost matrix must be provided")
else:
    # Define objective function
    self.model = PLP.LpProblem(name = self.name, sense =
    PLP.LpMinimize)
    self.model += PLP.lpSum(self.cost_matrix[i][j] * self.x[i][j]
                            for i in self.variable_range
                            for j in self.variable_range
                            ), "Objective"

    # Construct constraints according to the definition of the
    # assignment problem, which states that each job is assigned to
    # exactly one worker, and each worker is assigned to exactly one
    ↪ job.

    def construct_constraints(self):
        # Each job is assigned to exactly one worker
        for i in self.variable_range:
            self.model += PLP.lpSum(self.x[i][j] for j in self.variable_range)
            ↪ == 1, f"Job_{i}_constraint"
        # Each worker is assigned to exactly one job
        for j in self.variable_range:
            self.model += PLP.lpSum(self.x[i][j] for i in self.variable_range)
            ↪ == 1, f"Worker_{j}_constraint"
        # Positivity constraints are already defined by lowBound = 0 in
        ↪ variable definition
        self.constraints = "ADDED"

    def solve(self, quiet = True, positive_variables_only = True):
        if self.constraints != "ADDED":
            raise ValueError("Constraints must be ADDED")
        # Solve quietly
        print()
        self.model.solve(PLP.PULP_CBC_CMD(msg = 0 if quiet else 1))
        # Print af loesningens status

```

```

print("Status:", PLP.LpStatus[self.model.status])

# Print of values of the decision variables, with option to only print
→positive variables

if postive_variables_only:
    epsilon = 1e-5
    condition = lambda v: v.varValue > epsilon
else:
    condition = None
if condition is not None:
    for v in self.model.variables():
        if condition(v):
            print(v.name, "=", v.varValue)
else:
    for v in self.model.variables():
        print(v.name, "=", v.varValue)

# Print af den optimale objektfunktionsvaerdi
print("Value of Objective function. = ",
      PLP.value(self.model.objective))

def print_assignment_details(self, one_indexed = True):
    total_calculated_cost = 0
    oi = 1 if one_indexed else 0
    for i in self.variable_range:
        for j in self.variable_range:
            amount = self.x[i][j].varValue
            unit_cost = self.cost_matrix[i][j]
            route_cost = amount * unit_cost
            total_calculated_cost += route_cost
            print(f"Job {i + oi} is assigned to worker {j+ oi}, at cost_
→{route_cost}")

    print("Total calculated cost = ", total_calculated_cost)

```

```

[25]: distances = [150, 200, 250, 400, 500]
flows = [60, 50, 20, 90, 40]
plane_range = range(len(flows))
dist_range = range(len(flows))

cost_matrix_ap = np.zeros((len(plane_range), len(dist_range)))
for i in plane_range:
    for j in dist_range:
        cost_matrix_ap[i, j] = distances[j]*flows[i]
number_of_jobs = len(plane_range)
AP = AssignmentProblem(n = number_of_jobs, cost_matrix = cost_matrix_ap)

```

```
AP.construct_constraints()
AP.solve()
AP.print_assignment_details()
```

```
Status: Optimal
x_0_1 = 1.0
x_1_2 = 1.0
x_2_4 = 1.0
x_3_0 = 1.0
x_4_3 = 1.0
Value of Objective function. = 64000.0
Job 1 is assigned to worker 1, at cost 0.0
Job 1 is assigned to worker 2, at cost 12000.0
Job 1 is assigned to worker 3, at cost 0.0
Job 1 is assigned to worker 4, at cost 0.0
Job 1 is assigned to worker 5, at cost 0.0
Job 2 is assigned to worker 1, at cost 0.0
Job 2 is assigned to worker 2, at cost 0.0
Job 2 is assigned to worker 3, at cost 12500.0
Job 2 is assigned to worker 4, at cost 0.0
Job 2 is assigned to worker 5, at cost 0.0
Job 3 is assigned to worker 1, at cost 0.0
Job 3 is assigned to worker 2, at cost 0.0
Job 3 is assigned to worker 3, at cost 0.0
Job 3 is assigned to worker 4, at cost 0.0
Job 3 is assigned to worker 5, at cost 10000.0
Job 4 is assigned to worker 1, at cost 13500.0
Job 4 is assigned to worker 2, at cost 0.0
Job 4 is assigned to worker 3, at cost 0.0
Job 4 is assigned to worker 4, at cost 0.0
Job 4 is assigned to worker 5, at cost 0.0
Job 5 is assigned to worker 1, at cost 0.0
Job 5 is assigned to worker 2, at cost 0.0
Job 5 is assigned to worker 3, at cost 0.0
Job 5 is assigned to worker 4, at cost 16000.0
Job 5 is assigned to worker 5, at cost 0.0
Total calculated cost = 64000.0
```

We get the following setup:

Plane 1 is assigned to gate 2

Plane 2 is assigned to gate 3

Plane 3 is assigned to gate 5

Plane 4 is assigned to gate 1

Plane 5 is assigned to gate 4

This makes sense that 4 is placed at gate 1 since there is the largest flow from plane 4 and the smallest distance to the gate.

3.2 Spm. 2)

We cannot model this as an assignment problem because the cost of assigning a plane to one position interacts with the cost of the other planes. To model this we need the quadratic assignment problem (QAP). This is implemented in the class below:

```
[26]: import numpy as np
import pulp as PLP

class QuadraticAssignmentProblem:

    def __init__(self, machine_range, location_range, flow_matrix,
distance_matrix, max_capacities = None, name = "QuadraticAssignmentProblem"):
        # Or factory range
        self.machine_range = machine_range
        self.location_range = location_range
        self.flow_matrix = flow_matrix
        self.distance_matrix = distance_matrix
        self.max_capacities = max_capacities
        self.name = name
        # Constraint variables for linear formulation
        self.x = PLP.LpVariable.dicts("x",
                                     (self.machine_range, self.location_range),
                                     cat = PLP.LpBinary)

        # k > i, incodes that the distances between pairs of locations and
        # flows are symmetric.
        self.y_tuples = [(i,j,k,l)
                           for i in self.machine_range
                           for j in self.location_range
                           for k in self.machine_range
                           for l in self.location_range if k > i]

    def construct_model(self):
        self.y = PLP.LpVariable.dicts("y",
                                     self.y_tuples,
                                     cat=PLP.LpBinary)

        self.model = PLP.LpProblem(name=self.name, sense=PLP.LpMinimize)
        self.model += PLP.lpSum(self.flow_matrix[t[0]][t[2]] *
                                self.distance_matrix[t[1]][t[3]]
                                * self.y[t] for t in self.y_tuples), "Objective"
        self.model_construction = True

    def construct_constraints(self):
```

```

        # More machines than locations, more than one machine must be assigned
        to some locations.
        if len(self.machine_range) > len(self.location_range):
            if self.max_capacities is None:
                raise ValueError("If the number of machines is greater than the
                number of locations, max_capacities must be provided.")
            # Each machine is assigned to exactly one location
            for i in self.machine_range:
                self.model += PLP.lpSum(self.x[i][j] for j in self.
                location_range) == 1, f"Machine_{i}_constraint"
            # Each machine is assign to at most max_capacities[j] locations
            for j in self.location_range:
                self.model += PLP.lpSum(self.x[i][j] for i in self.
                machine_range) <= self.max_capacities[j], f"Location_{j}_constraint"

        # More locations than machines, so we can accept less than one machine
        at location
        elif len(self.location_range) > len(self.machine_range):
            # Each machine is assigned to exactly one location
            for i in self.machine_range:
                self.model += PLP.lpSum(self.x[i][j] for j in self.
                location_range) == 1, f"Machine_{i}_constraint"
            # Each location is assigned to at most one machine
            for j in self.location_range:
                self.model += PLP.lpSum(self.x[i][j] for i in self.
                machine_range) <= 1, f"Location_{j}_constraint"

        # Standard one to one case
        else:
            # Each machine is assigned to exactly one location
            for i in self.machine_range:
                self.model += PLP.lpSum(self.x[i][j] for j in self.
                location_range) == 1, f"Machine_{i}_constraint"
            # Each location is assigned to exactly one machine
            for j in self.location_range:
                self.model += PLP.lpSum(self.x[i][j] for i in self.
                machine_range) == 1, f"Location_{j}_constraint"

        # Other usual constraints
        for t in self.y_tuples:
            self.model += self.y[t] <= self.x[t[0]][t[1]]
            self.model += self.y[t] <= self.x[t[2]][t[3]]
            self.model += self.y[t] >= self.x[t[0]][t[1]] + self.x[t[2]][t[3]]

        self.constraints = "ADDED"

```

```

def solve(self, quiet = True, postive_variables_only = True):
    if not self.model_construction:
        raise Exception("You must construct the model before solving.")
    if self.constraints != "ADDED":
        raise Exception("You must constrain the model before solving.")
    # Solve quietly
    print()
    self.model.solve(PLP.PULP_CBC_CMD(msg = 0 if quiet else 1))
    # Print af loesningens status
    print("Status:", PLP.LpStatus[self.model.status])

    # Print of values of the decision variables, with option to only print
    ↪ positive variables

    if postive_variables_only:
        epsilon = 1e-5
        condition = lambda v: v.varValue > epsilon
    else:
        condition = None
    if condition is not None:
        for v in self.model.variables():
            if condition(v):
                print(v.name, "=", v.varValue)
    else:
        for v in self.model.variables():
            print(v.name, "=", v.varValue)

    # Print af den optimale objektfunktionsvaerdi
    print("Value of Objective function. = ",
          PLP.value(self.model.objective))

```

```

[ ]: ##### Spm. 1) #####

"""
I denne opgave ønsker vi at placere gates således at det vægtede flow
minimeres. Da der ingen interaktion er mellem flyene kan dette modelleres som
et normal assignment problem.
"""

gates = "A B C D E".split()
distances = [150, 200, 250, 400, 500]
flows = [60, 50, 20 , 90, 40]
n = 5
cost_matrix = np.zeros((n,n))

for i in range(n):

```



```

    for j in range(n):
        cost_matrix[i][j] = distances[j] * flows[i]
print("Cost matrix:", cost_matrix)

from modeller.network.assignment_problem import AssignmentProblem
variable_dict = dict((i,gates[j]) for i in range(n) for j in range(n))
assignment_problem = AssignmentProblem(n,
                                       cost_matrix = cost_matrix,
                                       name = "GateAssignment")

# Rename according to gate names.
for i in assignment_problem.variable_range:
    for j in assignment_problem.variable_range:
        assignment_problem.x[i][j].name = f"x_{i}_{gates[j]}"
assignment_problem.solve()

##### Spm. 2) #####

"""
Vi betragter nu en situation hvor kun kigger på inter-transit flow,
og ønsker at minimere det vægtede flow mellem gates. Dette er et QAP problem.
"""

from modeller.quadtratic_assignment_problem import QuadraticAssignmentProblem

distances = np.array([[0, 150, 200, 250, 400, 500],
                      [150, 0, 50, 100, 250, 350],
                      [200, 50, 0, 50, 300, 400],
                      [250, 100, 50, 0, 250, 350],
                      [400, 250, 300, 250, 0, 300],
                      [500, 350, 400, 350, 300, 0]])

flows = np.zeros((6, 6))

# upper triangle values
flows[0, 1:] = [60, 50, 20, 90, 40]
flows[1, 2:] = [10, 15, 2, 12]
flows[2, 3:] = [3, 20, 35]
flows[3, 4:] = [8, 11]
flows[4, 5:] = [9]

# make symmetric
flows = flows + flows.T

# Check symmetry
if all(distances[i][j] == distances[j][i] for i in range(n) for j in range(n)):
    print("Symmetric")
else:

```

```

    raise ValueError("Non-symmetric Matrix")

distances_no_gate = distances[1:,1:]
flows_no_gate = flows[1:,1:]

machine_range = range(n)
location_range = range(n)
QAD_no_gate = QuadraticAssignmentProblem(machine_range = machine_range,
                                          location_range = location_range,
                                          flow_matrix = flows_no_gate,
                                          distance_matrix = distances_no_gate)

# Rename according to gate names.
for i in QAD_no_gate.machine_range:
    for j in QAD_no_gate.location_range:
        QAD_no_gate.x[i][j].name = f"x_{i}_{gates[j]}"
QAD_no_gate.construct_model()
QAD_no_gate.construct_constraints()
QAD_no_gate.solve()

##### Spm. 3) #####
"""
We now also consider the traffic from the planes to the gates. This constitutes
a mixed AP-QAP problem, where we edit the objective in the QAP, such that
the linear AP is also considered.
"""

# Standard QAD
QAD = QuadraticAssignmentProblem(machine_range = machine_range,
                                location_range = location_range,
                                flow_matrix = flows_no_gate,
                                distance_matrix= distances_no_gate)

# Update the objective
QAD.construct_model()

QAD.model.objective = QAD.model.objective + PLP.lpSum(QAD.x[i][j]*
                                                    cost_matrix[i][j]
                                                    for i in range(n)
                                                    for j in range(n))

QAD.construct_constraints()

# Rename according to gate names.
for i in QAD.machine_range:
    for j in QAD.location_range:
        QAD.x[i][j].name = f"x_{i+1}_{gates[j]}"
print("QAP-AP")
QAD.solve()

```

```

print("QAP with dummy plane, forced to index 0")
n = n + 1
machine_range = range(n)
location_range = range(n)
QAD = QuadraticAssignmentProblem(machine_range = machine_range,
                                location_range = location_range,
                                flow_matrix = flows,
                                distance_matrix = distances)
# Zeroth index is a dummy plane. We must make the location of this plane fixed.
# Dummy plane is fixed to first gate, so we add the constraint that x[0][0] = 1
QAD.construct_model()
QAD.model += QAD.x[0][0] == 1, "DummyPlaneConstraint"
QAD.construct_constraints()
# Rename according to gate names.
gates = ["IU"] + gates
for i in QAD.machine_range:
    for j in QAD.location_range:
        QAD.x[i][j].name = f"x_{i}_{gates[j]}"
QAD.solve()

```

```

[51]: # Distance matrix
M = 10000
dist_mat = np.array([[M, 150, 200, 250, 400, 500],
                    [ 0, M, 50, 100, 250, 350],
                    [0, 0, M, 50, 300, 400],
                    [0,0,0, M, 250, 350],
                    [0,0,0,0, M, 300],
                    [0,0,0,0,0, M]])
# Symmetrize without doubling diagonal
dist_mat = dist_mat + dist_mat.T - np.diag(np.diag(dist_mat))

flow_mat = np.array([[M, 60, 50, 20, 90, 40],
                    [0, M, 10, 15, 2, 12],
                    [0, 0,M, 3, 20 , 35],
                    [0,0,0, M,8, 11],
                    [0,0,0,0,M,9],
                    [0,0,0,0,0, M]])
flow_mat = flow_mat + flow_mat.T - np.diag(np.diag(flow_mat))
print(dist_mat)
print(flow_mat)

```

```

[[10000  150   200   250   400   500]
 [  150 10000    50   100   250   350]
 [   200    50 10000    50   300   400]
 [   250   100    50 10000   250   350]
 [   400   250   300   250 10000   300]
 [   500   350   400   350   300 10000]]

```

```

[[10000    60    50    20    90    40]
 [   60 10000    10    15     2    12]
 [   50    10 10000     3    20    35]
 [   20    15     3 10000     8    11]
 [   90     2    20     8 10000     9]
 [   40    12    35    11     9 10000]]

```

```

[53]: # We dont consider gates, so we index the matrix row and column that assigns
      ↪ distance from gate to plane away

```

```

dist_mat2 = dist_mat[1:, 1:]
print(dist_mat2)
flow_mat2 = flow_mat[1:, 1:]
plane_range2 = plane_range
print(list(plane_range2))

QAP2 = QuadraticAssignmentProblem(plane_range2, plane_range2, flow_mat2,
      ↪ dist_mat2)
QAP2.construct_model()
QAP2.construct_constraints()
QAP2.solve()

```

```

[[10000    50    100    250    350]
 [   50 10000     50    300    400]
 [   100     50 10000    250    350]
 [   250    300    250 10000    300]
 [   350    400    350    300 10000]]
[0, 1, 2, 3, 4]

```

Status: Optimal

```

x_0_3 = 1.0
x_1_1 = 1.0
x_2_4 = 1.0
x_3_0 = 1.0
x_4_2 = 1.0
y_(0,_3,_1,_1) = 1.0
y_(0,_3,_2,_4) = 1.0
y_(0,_3,_3,_0) = 1.0
y_(0,_3,_4,_2) = 1.0
y_(1,_1,_2,_4) = 1.0
y_(1,_1,_3,_0) = 1.0
y_(1,_1,_4,_2) = 1.0
y_(2,_4,_3,_0) = 1.0
y_(2,_4,_4,_2) = 1.0
y_(3,_0,_4,_2) = 1.0
Value of Objective function. = 22500.0

```

3.3 Spm 3.)

To solve this, we can make a mixed QAP AP problem where we include a linear cost from the gates that depends on the assignment of the planes.

```
[55]: full_range = range(6)
QAP3 = QuadraticAssignmentProblem(plane_range, plane_range, flow_mat2,
    ↪dist_mat2)
QAP3.construct_model()
QAP3.model.objective = QAP3.model.objective + PLP.
    ↪lpSum(cost_matrix_ap[i][j]*QAP3.x[i][j] for i in plane_range for j in
    ↪plane_range)
QAP3.construct_constraints()
QAP3.solve()
```

Status: Optimal

x_0_3 = 1.0

x_1_1 = 1.0

x_2_4 = 1.0

x_3_0 = 1.0

x_4_2 = 1.0

y_(0,_3,_1,_1) = 1.0

y_(0,_3,_2,_4) = 1.0

y_(0,_3,_3,_0) = 1.0

y_(0,_3,_4,_2) = 1.0

y_(1,_1,_2,_4) = 1.0

y_(1,_1,_3,_0) = 1.0

y_(1,_1,_4,_2) = 1.0

y_(2,_4,_3,_0) = 1.0

y_(2,_4,_4,_2) = 1.0

y_(3,_0,_4,_2) = 1.0

Value of Objective function. = 90000.0